RaceDay API Endpoint Plan

1. Authentication

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Register a new user account	Public	FirstName, LastName, Email, Password, Role	201 Created; 400 Bad Request; 409 Conflict
POST	/api/auth/login	Authenticate a user and start a session	Public	Email, Password	200 OK; 400 Bad Request; 401 Unauthorized
POST	/api/auth/logout	End the current user session	Participant/Organiser	None	200 OK; 401 Unauthorized

2. User Profile

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/profile	View the currently logged-in user's profile	Participant/Organiser	None	200 OK; 401 Unauthorized; 404 Not Found
PUT	/api/users/profile	Update the currently logged-in user's profile	Participant/Organiser	FirstName, LastName, Email, PhoneNumber, DateOfBirth, ProfilePictureUrl	200 OK; 400 Bad Request; 401 Unauthorized

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/{id}	View a user's basic information	Organiser	None	200 OK; 401 Unauthorized; 404 Not Found

3. Events

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	View available events	Participant	None	200 OK; 401 Unauthorized
GET	/api/events/{id}	View details of a specific event	Participant/Organiser	None	200 OK; 401 Unauthorized; 404 Not Found
POST	/api/events	Create a new event	Organiser	Name, Description, EventDate, Location, Distance, EventType	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden
PUT	/api/events/{id}	Edit an existing event	Organiser	Name, Description, EventDate, Location, Distance, EventType	200 OK; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found
DELETE	/api/events/{id}	Delete an event	Organiser	None	204 No Content; 401 Unauthorized;

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
					403 Forbidden; 404 Not Found

4. Categories

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	View categories belonging to an event	Participant/Organiser	None	200 OK; 401 Unauthorized; 404 Not Found
POST	/api/events/{eventId}/categories	Create a category for an event	Organiser	Name, Description	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found
PUT	/api/categories/{id}	Edit an event category	Organiser	Name, Description	200 OK; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
DELETE	/api/categories/{id}	Delete an event category	Organiser	None	204 No Content; 401 Unauthorized; 403 Forbidden; 404 Not Found

5. Enrolments

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/events/{eventId}/enrolments	Enter the logged-in participant into an event category	Participant	CategoryId	201 Created; 400 Bad Request; 401 Unauthorized; 404 Not Found; 409 Conflict
GET	/api/enrolments/my	View the logged-in participant's enrolments	Participant	None	200 OK; 401 Unauthorized
GET	/api/events/{eventId}/enrolments	View all enrolments for an event	Organiser	None	200 OK; 401 Unauthorized; 403 Forbidden;

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
					404 Not Found
GET	/api/enrolments/{id}	View a specific enrolment	Participant/Organiser	None	200 OK; 401 Unauthorized; 403 Forbidden; 404 Not Found
DELETE	/api/enrolments/{id}	Cancel an enrolment	Participant	None	204 No Content; 401 Unauthorized; 403 Forbidden; 404 Not Found

6. Results

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments/{enrolmentId}/result	Record a participant's result	Organiser	FinishTime, FinishingPosition	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found
GET	/api/results/my	View the logged-in	Participant	None	200 OK; 401

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
		participant's results			Unauthorized
GET	/api/events/{eventId}/results	View results for an event	Organiser	None	200 OK; 401 Unauthorized; 403 Forbidden; 404 Not Found
GET	/api/results/{id}	View a specific result	Participant/Organiser	None	200 OK; 401 Unauthorized; 403 Forbidden; 404 Not Found
PUT	/api/results/{id}	Update a participant's result	Organiser	FinishTime, FinishingPosition	200 OK; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found
DELETE	/api/results/{id}	Remove a result	Organiser	None	204 No Content; 401 Unauthorized; 403 Forbidden; 404 Not Found

```
-- ============================================================
-- RaceDay Database
-- Part 1 - System Planning and Database
-- SQL Server
-- ============================================================

-- Create database
IF DB_ID('RaceDay') IS NULL
BEGIN
CREATE DATABASE RaceDay;
END
GO

USE RaceDay;
GO

-- ============================================================
-- DROP EXISTING TABLES
-- Allows the script to be re-run during development.
-- ============================================================

IF OBJECT_ID('dbo.Results', 'U') IS NOT NULL
DROP TABLE dbo.Results;
GO

IF OBJECT_ID('dbo.Enrolments', 'U') IS NOT NULL
DROP TABLE dbo.Enrolments;
GO

IF OBJECT_ID('dbo.Categories', 'U') IS NOT NULL
DROP TABLE dbo.Categories;
GO

IF OBJECT_ID('dbo.Events', 'U') IS NOT NULL
DROP TABLE dbo.Events;
GO

IF OBJECT_ID('dbo.UserProfiles', 'U') IS NOT NULL
DROP TABLE dbo.UserProfiles;
GO

IF OBJECT_ID('dbo.Users', 'U') IS NOT NULL
DROP TABLE dbo.Users;
GO
```

```
-- ============================================================
-- USERS
-- ============================================================

CREATE TABLE Users
(
UserId INT IDENTITY(1,1) NOT NULL,
FirstName NVARCHAR(50) NOT NULL,
LastName NVARCHAR(50) NOT NULL,
Email NVARCHAR(255) NOT NULL,
PasswordHash NVARCHAR(255) NOT NULL,
Role NVARCHAR(20) NOT NULL,

CONSTRAINT PK_Users

  PRIMARY KEY (UserId),


CONSTRAINT UQ_Users_Email

  UNIQUE (Email),


CONSTRAINT CK_Users_Role

  CHECK (Role IN ('Organiser', 'Participant'))

);
GO

-- ============================================================
-- USER PROFILES
-- One profile belongs to one user.
-- ============================================================

CREATE TABLE UserProfiles
(
ProfileId INT IDENTITY(1,1) NOT NULL,
UserId INT NOT NULL,
PhoneNumber NVARCHAR(30) NULL,
DateOfBirth DATE NULL,
ProfilePictureUrl NVARCHAR(500) NULL,

CONSTRAINT PK_UserProfiles

  PRIMARY KEY (ProfileId),
```

```
CONSTRAINT UQ_UserProfiles_UserId
  UNIQUE (UserId),

CONSTRAINT FK_UserProfiles_Users
  FOREIGN KEY (UserId)
  REFERENCES Users(UserId)
  ON DELETE CASCADE
);
GO

-- ============================================================
-- EVENTS
-- Each event is created by an organiser.
-- ============================================================
CREATE TABLE Events
(
EventId INT IDENTITY(1,1) NOT NULL,
OrganiserId INT NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(1000) NULL,
EventDate DATETIME2 NOT NULL,
Location NVARCHAR(255) NOT NULL,
Distance DECIMAL(6,2) NOT NULL,
EventType NVARCHAR(20) NOT NULL,
CONSTRAINT PK_Events
  PRIMARY KEY (EventId),

CONSTRAINT FK_Events_Users
  FOREIGN KEY (OrganiserId)
  REFERENCES Users(UserId),

CONSTRAINT CK_Events_Distance
```

```
  CHECK (Distance > 0),


CONSTRAINT CK_Events_EventType
  CHECK (EventType IN ('Run', 'Walk', 'Cycle'))
);
GO

-- ============================================================
-- CATEGORIES
-- Each category belongs to one event.
-- ============================================================

CREATE TABLE Categories
(
CategoryId INT IDENTITY(1,1) NOT NULL,
EventId INT NOT NULL,
Name NVARCHAR(100) NOT NULL,
Description NVARCHAR(500) NULL,

CONSTRAINT PK_Categories
  PRIMARY KEY (CategoryId),


CONSTRAINT FK_Categories_Events
  FOREIGN KEY (EventId)
  REFERENCES Events(EventId)
  ON DELETE CASCADE
);
GO

-- ============================================================
-- ENROLMENTS
-- Connects participants to events and categories.
-- ============================================================

CREATE TABLE Enrolments
(
EnrolmentId INT IDENTITY(1,1) NOT NULL,
ParticipantId INT NOT NULL,
```

```
EventId INT NOT NULL,
CategoryId INT NOT NULL,
EnrolmentDate DATETIME2 NOT NULL
CONSTRAINT DF_Enrolments_EnrolmentDate
DEFAULT SYSUTCDATETIME(),
Status NVARCHAR(20) NOT NULL
CONSTRAINT DF_Enrolments_Status
DEFAULT 'Active',

CONSTRAINT PK_Enrolments

  PRIMARY KEY (EnrolmentId),


CONSTRAINT FK_Enrolments_Participant

  FOREIGN KEY (ParticipantId)

  REFERENCES Users(UserId),


CONSTRAINT FK_Enrolments_Event

  FOREIGN KEY (EventId)

  REFERENCES Events(EventId),


CONSTRAINT FK_Enrolments_Category

  FOREIGN KEY (CategoryId)

  REFERENCES Categories(CategoryId),


CONSTRAINT CK_Enrolments_Status

  CHECK (Status IN ('Active', 'Cancelled')),


CONSTRAINT UQ_Enrolments_Participant_Event_Category

  UNIQUE (ParticipantId, EventId, CategoryId)

);
GO
```

```
-- ============================================================
-- RESULTS
-- An enrolment can have zero or one result.
-- ============================================================

CREATE TABLE Results
(
ResultId INT IDENTITY(1,1) NOT NULL,
EnrolmentId INT NOT NULL,
FinishTime TIME NOT NULL,
FinishingPosition INT NOT NULL,

CONSTRAINT PK_Results

  PRIMARY KEY (ResultId),


CONSTRAINT UQ_Results_EnrolmentId

  UNIQUE (EnrolmentId),


CONSTRAINT FK_Results_Enrolments

  FOREIGN KEY (EnrolmentId)

  REFERENCES Enrolments(EnrolmentId)

  ON DELETE CASCADE,


CONSTRAINT CK_Results_FinishingPosition

  CHECK (FinishingPosition > 0)

);
GO

-- ============================================================
-- SEED USERS
-- Minimum required: 2 Organisers and 2 Participants.
-- PasswordHash values are placeholders for Part 1 database
-- planning. Part 2 will implement proper password hashing.
-- ============================================================

INSERT INTO Users
(FirstName, LastName, Email, PasswordHash, Role)
```

```
VALUES
('John', 'Smith', 'john.organiser@raceday.co.za', 'HASH_PLACEHOLDER_1', 'Organiser'),
('Sarah', 'Naidoo', 'sarah.organiser@raceday.co.za', 'HASH_PLACEHOLDER_2',
'Organiser'),
('Michael', 'Jones', 'michael.participant@raceday.co.za', 'HASH_PLACEHOLDER_3',
'Participant'),
('Thandi', 'Mokoena', 'thandi.participant@raceday.co.za', 'HASH_PLACEHOLDER_4',
'Participant');
GO

-- ============================================================
-- SEED USER PROFILES
-- ============================================================

INSERT INTO UserProfiles
(UserId, PhoneNumber, DateOfBirth, ProfilePictureUrl)
VALUES
(1, '0825551001', '1990-04-15', NULL),
(2, '0825551002', '1988-09-22', NULL),
(3, '0825551003', '2002-02-10', NULL),
(4, '0825551004', '2001-11-30', NULL);
GO

-- ============================================================
-- SEED EVENTS
-- Minimum required: 3 events.
-- ============================================================

INSERT INTO Events
(OrganiserId, Name, Description, EventDate, Location, Distance, EventType)
VALUES
(
1,
'Cape Town Road Run',
'A road running event through Cape Town.',
'2027-03-14 07:00:00',
'Cape Town',
10.00,
'Run'
),
(
1,
'Table Mountain Walk',
```

```
'A recreational walking event.',
'2027-04-10 08:00:00',
'Cape Town',
8.00,
'Walk'
),
(
2,
'Cape Cycle Challenge',
'A cycling event for participants of different experience levels.',
'2027-05-22 06:30:00',
'Cape Town',
42.00,
'Cycle'
);
GO

-- ============================================================
-- SEED CATEGORIES
-- Each event receives categories.
-- ============================================================

INSERT INTO Categories
(EventId, Name, Description)
VALUES
(1, 'Open 10km', '10 kilometre open category.'),
(1, 'Junior 10km', '10 kilometre junior category.'),
(2, 'Open Walk', 'Open recreational walking category.'),
(2, 'Junior Walk', 'Junior walking category.'),
(3, 'Open Cycle', 'Open cycling category.'),
(3, 'Junior Cycle', 'Junior cycling category.');
GO

-- ============================================================
-- SEED ENROLMENTS
-- Sample participant enrolments.
-- ============================================================

INSERT INTO Enrolments
(ParticipantId, EventId, CategoryId, Status)
VALUES
(3, 1, 1, 'Active'),
(4, 1, 1, 'Active'),
```

```
(3, 2, 3, 'Active'),
(4, 3, 5, 'Active');
GO

-- ============================================================
-- SEED RESULTS
-- Results are associated with existing enrolments.
-- ============================================================

INSERT INTO Results
(EnrolmentId, FinishTime, FinishingPosition)
VALUES
(1, '00:52:35', 12),
(2, '00:55:18', 18);
GO

-- ============================================================
-- VERIFICATION QUERIES
-- ============================================================

SELECT * FROM Users;
SELECT * FROM UserProfiles;
SELECT * FROM Events;
SELECT * FROM Categories;
SELECT * FROM Enrolments;
SELECT * FROM Results;
GO
```